LaserIon Project

CINECA HPC + AI summer school 2026

Giovanni
Flavia
Olivier
Eduardo

What laserion Computes

Tunnel ionization by an intense laser field, mapped to electron momentum

Laser pulse definition

Temporal envelope, transverse profile, and polarization state; single- and multi-pulse superposition.

ADK ionization model

Instantaneous tunneling rate w(|E|) for each species and charge state Z, evaluated along the field history.

Momentum distribution (MDF)

Ionization time -> drift momentum via the vector potential, weighted by ionization probability.

Macroparticle sampling

Cells are populated with weighted (or equal-weight) macroparticles for diagnostics and HDF5 output.

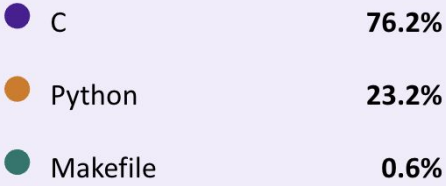
Repository Layout

Two parallel implementations sharing one physics model

laserion/

```
  src/laserion/     — Python reference package (core.py, mdf.py, ionization.py)
  csrc/               — Production C implementation (MPI + HDF5)
    src/            — 16 source files (laser, ionization, mdf, diagnostics)
      include/          — public headers
      vendor/           — bundled tomlc99 (TOML parser)
      inputdeck/        — example .toml run configurations
  examples/           — examples.py (Python usage)
  tests/              — Python vs. C cross-validation scripts
  docs/               — test/validation notes (adk, pulse,
multipulse...)
```

Language mix



C build requires an MPI compiler (mpicc) + HDF5; recommended for production runs.

C version already parallelized with **MPI**

Step 1: profiling base implementation

1) Instrument code with **score-p**
2) Run **scalasca -analyze laserion**
3) Profiling results visualization with **cube**

Step 1: profiling base implementation

Main bottlenecks:

- MPI_Barrier
- HDF5 file I/O (Hierarchical Data Format, used as cache and output files by the code)

Step 2: HDF5 I/O improvements (I)

ORIGINAL CODE

mdf_particles.c

cache_read_component_timeseries()

was called 6 times (Ex, Ey, Ez, Ax, Ay, Az)

for each particle

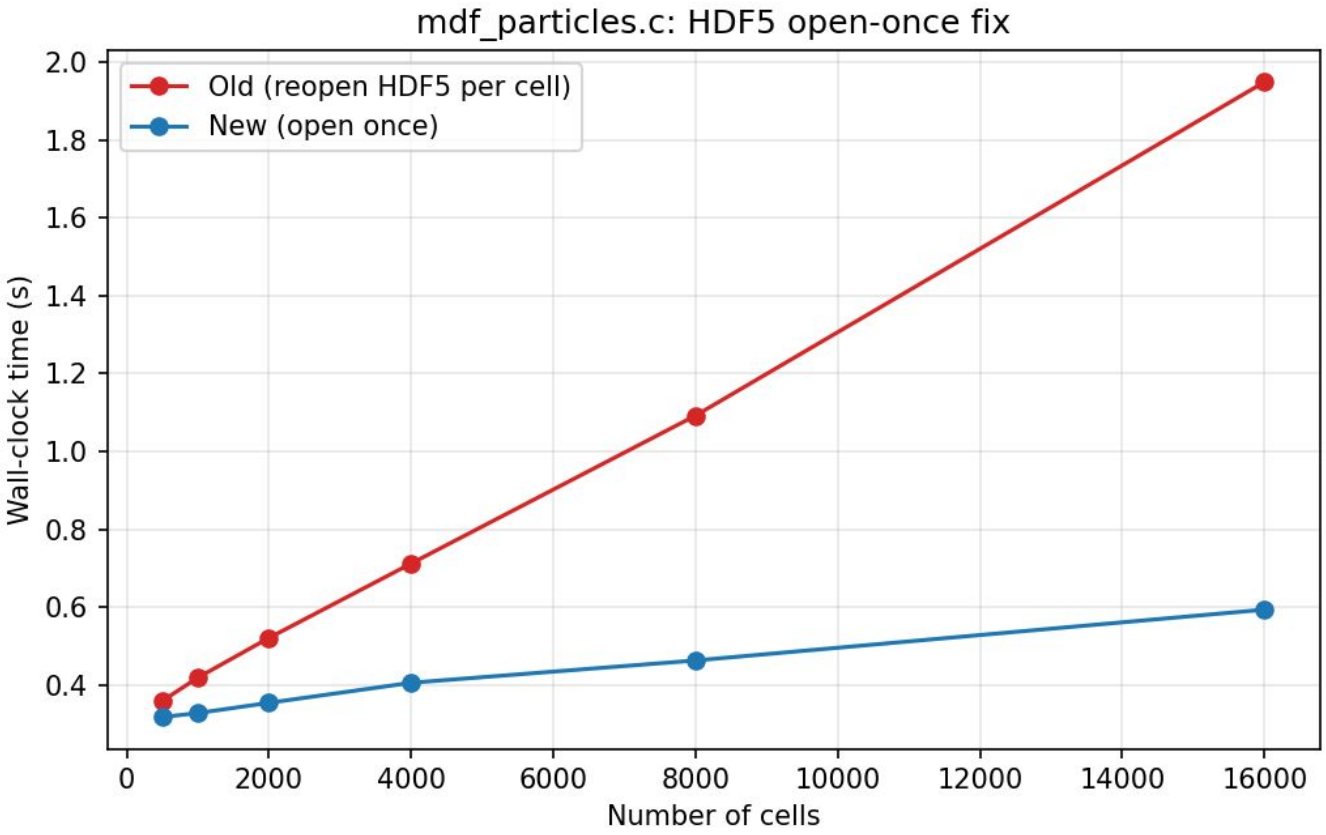
Each call is opening HDF5 file, reading and closing the file.

OUR CODE

mdf_particles.c

Read HDF5 file once before the loop and cache data in CacheComp data structure.

Later (inside loop) the data is read from cache using cachecomp_read_point().


mdf_particles.c: HDF5 open-once fix
Old (reopen HDF5 per cell)
New (open once)
Wall-clock time (s)
Number of cells
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
2000
4000
6000
8000
10000
12000
14000
16000

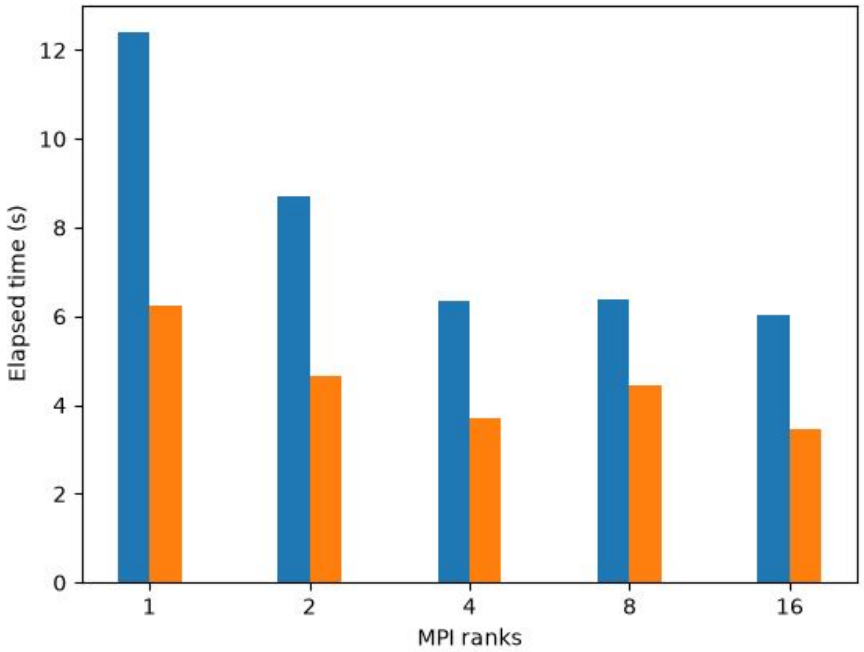
Step 2: HDF5 I/O improvements (II)

field_cache.c

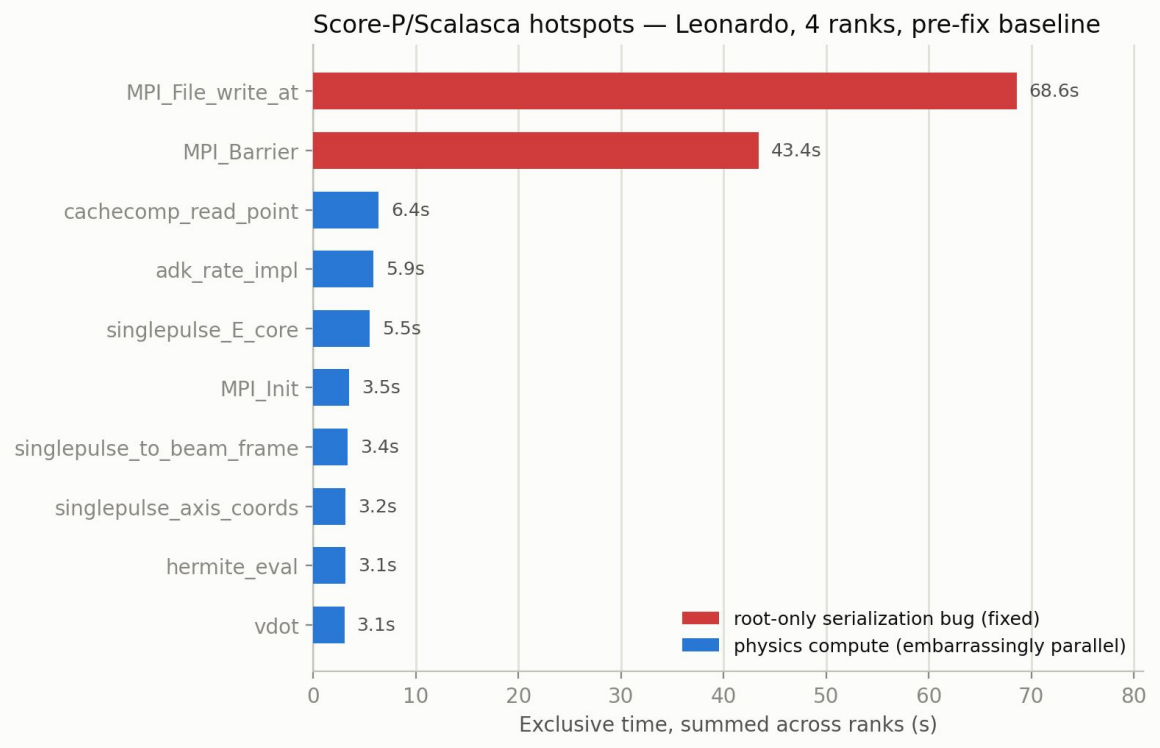
create_single_dataset_file()

swap time and space dimensions in writing (time becomes innermost loop dimension) so that each MPI rank writes data that is contiguous in memory (row-wise instead of column-wise)

Step 3: scaling tests on new code



Step 3 : profiling the code



Fixing MPI_Barrier()

- Both diagnostics ran root-only: rank 0 did all the work while ranks 1-3 sat idle at an MPI_Barrier

- Fix: round-robin job distribution in field_diag.c, spatial decomposition + MPI_Gatherv in ionization_diag.c

Fixing MPI_Barrier()